

Shot Clustering Techniques for Story Browsing

Wallapak Tavanapong, *Member, IEEE*, and Junyu Zhou

Abstract—Automatic video segmentation is the first and necessary step for organizing a long video file into several smaller units. The smallest basic unit is a shot. Relevant shots are typically grouped into a high-level unit called a scene. Each scene is part of a story. Browsing these scenes unfolds the entire story of a film, enabling users to locate their desired video segments quickly and efficiently.

Existing scene definitions are rather broad, making it difficult to compare the performance of existing techniques and to develop a better one. This paper introduces a stricter scene definition for narrative films and presents *ShotWeave*, a novel technique for clustering relevant shots into a scene using the stricter definition. The crux of *ShotWeave* is its feature extraction and comparison. Visual features are extracted from selected regions of representative frames of shots. These regions capture essential information needed to maintain viewers' thought in the presence of shot breaks. The new feature comparison is developed based on common continuity-editing techniques used in film making. Experiments were performed on full-length films with a wide range of camera motions and a complex composition of shots. The experimental results show that *ShotWeave* outperforms two recent techniques utilizing global visual features in terms of segmentation accuracy and time.

Index Terms—Content-based indexing and retrieval, feature extraction, scene segmentation, video browsing.

I. INTRODUCTION

RAPID ADVANCES in multimedia processing, computing power, high-speed internetworking, and the World-Wide Web have made digital videos an important part of many emerging applications such as distance learning, digital libraries, and electronic commerce. Searching for a desired video segment from a large collection of videos becomes increasingly more difficult as more digital videos are easily created. A well-known search approach matching user-specified keywords with titles, subjects, or short text descriptions is not effective because these descriptions are too coarse to capture rich semantics inherent in most videos. As a result, a long list of search results is expected. Users pinpoint their desired video segment by watching each video from the beginning or skimming through the video using fast-forward and fast-reverse operations.

Content-based video browsing and retrieval is an alternative that lets users browse and retrieve desired video segments in a nonsequential fashion. *Video segmentation* divides a video file into *shots* defined as a contiguous sequence of video frames

recorded from a single camera operation [1]–[3]. More meaningful high-level aggregates of shots are then generated for browsing and retrieval. This is because 1) users are more likely to recall important events rather than a particular shot or frame [4]; and 2) the number of shots in a typical film is too large for effective browsing (e.g., about 600–1500 shots for a typical film [5]).

Since manual segmentation is very time consuming (i.e., 10 h of work for 1 h of video [5]), recent years have seen a plethora of research on automatic video segmentation techniques. A typical automatic video segmentation involves three important steps. The first step is *shot boundary detection* (SBD). A shot boundary is declared if a dissimilarity measure between consecutive frames exceeds a threshold. Examples of recent SBD techniques are [1]–[3], [6]–[11]. The second step is *keyframe selection* that extracts one or more frames that best represent the shot, termed keyframe(s) for each shot. Recent techniques include [2], [3], [12]–[15]. *Scene segmentation* is the final step that groups related shots into a meaningful high-level unit termed a *scene*¹ in this paper. We focus on this step for a narrative film—a film that tells a story [16]. Viewers understand a complex story through the identification of important events and the association of these events by cause and effect, time, and space. Most movies are narrative.

A. Challenges in Automatic Scene Segmentation

Since a scene is based on human understanding of the meaning of a video segment, it is very difficult to give an objective and concise scene definition that covers all possible scenes judged by humans. This and the fact that benchmarks of video databases do not exist make it difficult to evaluate existing scene segmentation techniques and to develop a better one. A scene definition found in the literature is a sequence of shots unified by a common locale or an individual event [17]–[20]. Another scene definition also includes parallel events [4]. That is, two or more events are considered parallel if they happen simultaneously in the story time. For instance, in the movie *Home Alone*, one important scene consists of two parallel events: 1) the whole family is leaving the house by plane, and 2) Kevin, the main character, is left behind alone in his family home. The film director conveys the fact that the two events happen simultaneously in the story time by crosscutting² these events together (i.e., the shot of the plane taking off is followed by the shot of Kevin walking downstairs and is followed by the shot of the plane in the air and so on). Nevertheless, it is not clear what constitutes an event in these definitions. For instance, when several events happen in the same locale, should

Manuscript received February 20, 2002; revised November 8, 2002. This work was supported in part by the National Science Foundation under Grant CCR 0092914. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Mauro Barni.

W. Tavanapong is with the Department of Computer Science, Iowa State University, Ames, IA 50011-1040 USA (e-mail: tavanapo@cs.iastate.edu).

J. Zhou is with the Department of Computer Science, Iowa State University, Ames, IA 50011-1040 USA.

Digital Object Identifier 10.1109/TMM.2004.830810

¹The term “scene” has been used to mean shots in some publications.

²“Editing that alternates shots of two or more lines of actions occurring in different places, usually simultaneously [16].”

each event be considered a scene or should they all belong to the same scene?

Existing scene segmentation techniques can be divided into two categories: one using only visual features (e.g., [4], [18]–[22]) and the other using both visual and audio features (e.g., [23]–[25]). In both categories, visual similarities of entire shots or keyframes (i.e., global color histograms or color moments) are used for clustering shots into scenes. That is, global visual features of nearby shots are compared. If the dissimilarity measure of the features representing the shots is within the threshold, these shots and the shots in between them are considered in the same scene. Global features, however, tend to be too coarse for shot clustering because they include noise—objects that are excluded when humans group shots into scenes. Determining the appropriate areas of video frames (or objects) to use and when to use which area (objects) for correct shot clustering is challenging even if objects can be reliably recognized using very advanced object recognition techniques.

B. Our Approach and Contributions

In this paper, we first introduce a stricter scene definition for narrative films based on selected continuity-editing techniques in film literature. Many narrative films are available today. The definition is not applicable to advertisements, sports, or news video clips that have been specifically addressed by other recent work. We define scene this way because these editing techniques have been used in many films to successfully convey stories to most viewers regardless of the stories in the films. Compared to existing definitions, our strict definition is less subjective and should give scenes that are familiar to most viewers when browsing and querying. Second, we propose a novel shot clustering technique called *ShotWeave* that detects scenes based on the strict definition. Although *ShotWeave* currently uses only visual features like some existing techniques, its uniqueness is as follows. It extracts features from two predetermined areas of keyframes instead of the entire keyframes. These regions are carefully selected to capture essential information needed to maintain viewers' thought in the presence of shot breaks and to reduce noise that often confuses existing techniques. The extracted features are utilized in several steps as guided by the strict scene definition and editing techniques to lessen the possibility of wrongly separating shots of the same scene. Finally, we implement *ShotWeave* and two recent scene segmentation techniques. Both recent techniques use global features and were shown to perform well for movies. We evaluate the performance of the three techniques through experiments on two full-length films, each lasting more than 100 min. Our experimental results show that *ShotWeave* gives better segmentation accuracy and is faster than the two techniques.

The remainder of this paper is organized as follows. In Section II, we summarize the recent techniques. The strict scene definition and *ShotWeave* are presented in detail in Section III. In Section IV, the experimental study of the three techniques is presented. Section V offers our concluding remarks.

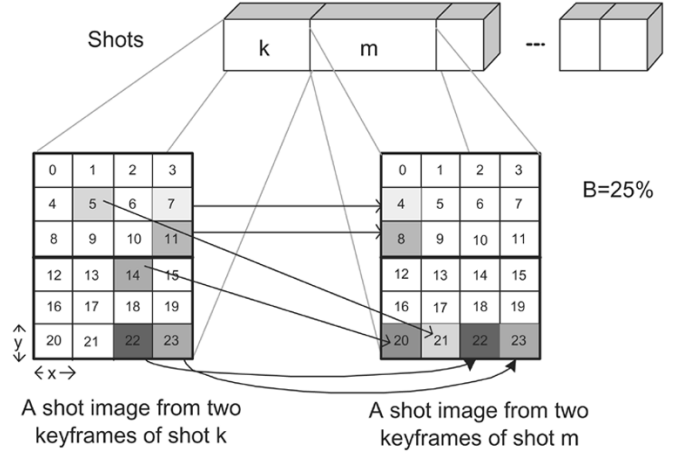


Fig. 1. SIM: comparison of shot images.

II. SHOT CLUSTERING TECHNIQUES USING GLOBAL FEATURES

Two recent techniques are summarized in this section. Due to lack of the original names given by the original authors, these techniques are referred as *Shot Image Matching* (SIM) for the work by Hanjalic *et al.* [4] and *Table-of-Content* (ToC) for the technique by Rui *et al.* [18].

A. Shot Image Matching

Given shot boundaries detected using a SBD technique, Hanjalic *et al.* employ shot images of detected shots to approximate scenes [4]. A shot image is a concatenation of all keyframes of a shot and is further divided into blocks of $x \times y$ pixels, as shown in Fig. 1. Shot images are compared during the clustering process using three important parameters: the percentage of the best matching blocks to the total blocks in the shot image (B), the forward search range (F), and the backward search range (R). The dissimilarity between any two shots is the average Euclidean distance of the average color (in LUV color space) between B best matching blocks of the corresponding shot images. Starting from the first shot, the current shot is compared with at most F subsequent shots to find the nearest matching shot (i.e., the dissimilarity between the shot images of these shots is within the allowed threshold). The threshold is adaptive based on the accumulative dissimilarity of shots included in the scene so far. If the matching shot is found, it becomes the current shot, and the same procedure repeats. Otherwise, at most, R preceding shots of the current shot are tested whether any of them matches with their F subsequent shots. Unless a matching shot is found, a scene cut is declared, and the next shot becomes the current shot. Note that whenever a matching pair is found, the pair and the shots in between are automatically assigned to the same scene.

The effectiveness of SIM relies heavily on the parameter B , which was experimentally determined in [4]. When B is large (i.e., many blocks are used in the dissimilarity calculation), SIM tends to separate shots of the same scene. With small B , SIM is likely to combine unrelated shots into the same scene. In general, SIM tends to combine shots of different scenes into the same scene because the technique searches for the best matching blocks between two shot images without taking semantics into

account. Furthermore, it is also difficult to determine the value of B that covers the different ways humans group shots into scenes.

B. ToC

ToC organizes shots into groups and then scenes as shown in Fig. 2. For each shot, ToC computes 1) color histograms of the entire keyframes of the shot and 2) *shot activity* calculated as the average color difference of all the frames in the shot. Low shot activity means that very small movements occur within a shot. The similarity measure of any two shots is the weighted sum of both the difference of the shot activities and the maximum color difference of the keyframes of the shots. Histogram intersection of global color histograms is used for calculating the color difference. Distance between shots is also considered in the computation of the difference in shot activities.

ToC clusters shots as follows. Starting from the first shot, the current shot is assigned to its most similar group if their similarity is at least a predetermined *group threshold*. In this case, the shots between the current shot and the last shot of the group are automatically assigned to the same scene. However, if no groups are sufficiently similar to the current shot, a new group having only this shot is created. The new group is assigned to its most similar scene if the similarity measure between the shot and the average similarity of all the groups in the scene is at least a predetermined *scene threshold*. Otherwise, a new scene is created for this group. Subsequent shots are considered similarly. In Fig. 2, shot 0 is initially assigned to group 0 and scene 0. Shot 1 is assigned to a new group (group 1) and a new scene (scene 1) since the shot is not similar to group 0. However, shot 2 is assigned to group 0 due to their similarity, causing the re-assignment of group 1 to scene 0 and the removal of scene 1. Shot 3 is assigned to group 1 due to their similarity. Since shot 4 is not similar to any existing group, a new group (group 2) and a new scene (scene 1) are created for the shot.

Our experiments indicate that ToC tends to generate more false scenes by separating shots of the same scene since noise—visual information of objects irrelevant in shot clustering is included in the global features.

III. STRICT SCENE DEFINITION AND SHOTWEAVE

In this section, we first provide a background on continuity editing techniques that were developed to create a smooth flow of viewers' thoughts from shot to shot in film literature [16]. We then discuss the strict scene definition, the feature extraction and comparison, and the clustering algorithm.

A. Continuity Editing Techniques

Continuity editing provides temporal continuity and spatial continuity. Temporal continuity refers to the presentation of shots in the order that events take place in the story time. Three commonly used techniques providing spatial continuity are as follows.

- **The 180° system.** All cameras are positioned on only one side of an imaginary line called the 180° line. In Fig. 3, a sequence of shots 1, 2, and 3 indicates that two people are

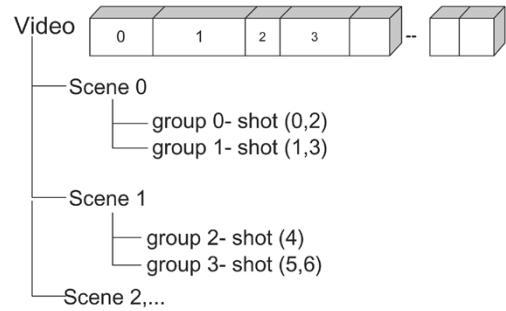


Fig. 2. Shot clustering using ToC.

walking toward each other. Assume that a blue building is behind them at a distance away. The 180° system ensures the following.

- A common space between consecutive shots, indicating that they are in the same locale. In Fig. 3, shots 1, 2, and 3 share a common background—the blue building.
- The location and the movements of the characters in relation to one another. In Fig. 3, person A is to the left of person B, and both are moving toward each other. If shot 2 is replaced by shot 2a (i.e., a camera is on the other side of the 180° line), which violates the 180° system, viewers no longer see the blue building and see that A is not facing B. This would cause the viewers to think that A is no longer walking toward B.
- **Shot/reverse-shot.** Once the 180° line is established, shots of each end point of the line can be interleaved since viewers have learned the locations of the characters from the previous shots. Typical examples of shot/reverse-shot involve conversations between characters. That is, a shot focusing on one character is interleaved with another shot capturing another character; the next shot cuts back to the first character, and so forth. Alternating closeup shots of A and B in Fig. 3 shows shot/reverse-shot. Shot/reverse-shot can also describe an interaction between a character and objects of interest.
- **Establishment/breakdown/re-establishment.** Establishment consists of establishing shot(s). It indicates the overall space, introduces main characters, and establishes the 180° line. The breakdown gives more details about what happens with the characters and is typically described using shot/reverse-shots. The re-establishment consisting of re-establishing shot(s) describes the overall space or participating characters again. For instance, shot 1 in Fig. 3 functions as an establishment, and shot 5 is a re-establishment.

Film directors sometimes violate continuity rules. For instance, flashbacks into the past or flashforward into the future violate the temporal continuity. However, a cut back to the present time is typically used since the user must know when the event happens in relation to the present. This creates a similar effect as shot/reverse-shot. Other editing techniques, such as matching of graphics between shots or manipulation of shot lengths to convey certain feelings, do not affect the construction of the story.

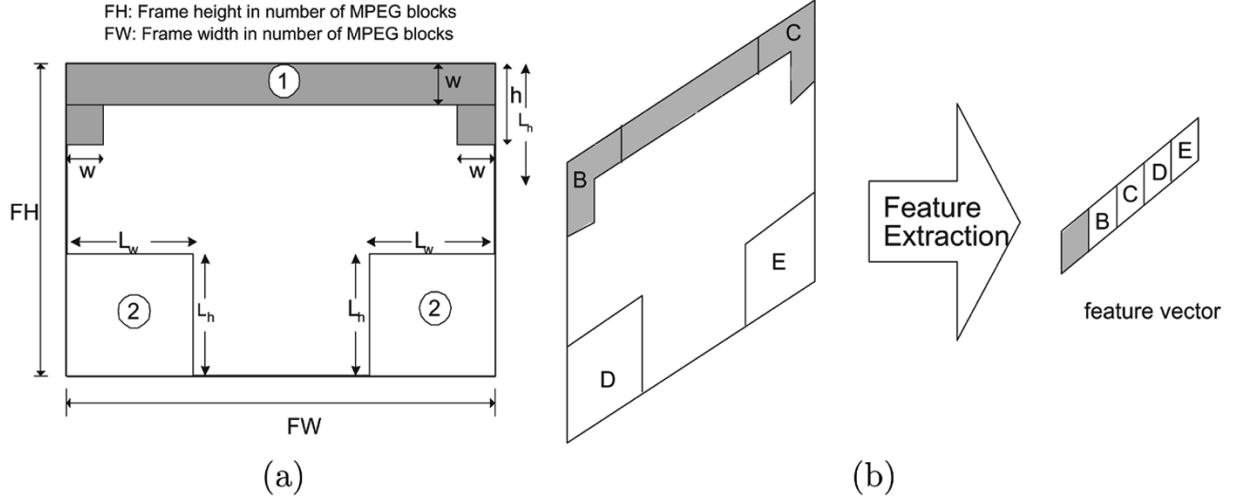


Fig. 4. Selected regions and feature extraction. (a) Selected regions and (b) feature extraction.

event. An event of a particular type can be labeled with more details, for instance, “an explosion event” by analyzing other media such as captions or audio.

The strict scene definition is given in Table II. The traveling-scene type indicates a significant change in locations and/or times between neighboring scenes. Within the traveling scene, the traveler is most important, not locales because the character only passes through each locale very briefly. If there were to be an important story in each place, an interacting event is needed to explain the story. In this case, the serial-event scene should be detected instead. Film directors may use other techniques to indicate a significant change in locations and/or times such as inserting the name of the location or the story time in the establishing shot of the next scene.

Most scenes belong to the serial-event type. When both establishment and re-establishment are omitted, the scene consists of only the interacting event. An example of a serial-event scene follows. The establishing shot is a wide-angle shot capturing three people at a dining table. The interacting event is the conversation among them conveyed by the 180° system and shot/reverse-shot. Finally, the re-establishing shot recaptures the three people at the same dining table. Flashbacks or flashforward can also be captured in this event type since the character interacts with objects of interest (i.e., the past or the future). The *Home Alone* scene mentioned previously is a good example of a parallel-event scene. Compared to the existing definitions, the strict definition is more objective and should result in scenes familiar to most viewers. Although we do not see a clear advantage of defining more scene types, we see the possibility of a scene that almost belongs to a certain type. However, the scene does not exactly satisfy the definition of the type due to some tricks chosen by the film directors. For instance, a parallel-event scene may consist of one serial-event scene interleaved with noninteracting events. In other words, an interacting event is missing from the second serial-event scene. In this case, it is recommended to consider the scene as belonging to its closest type. Note that the strict scene definition is not based on low-level features. Hence, it needn't be changed if other media types such as audio are used in determining scenes.

The detected scene type coupled with additional processing of audio or captions will be useful for generating basic annotations for scenes in the future. For example, for the traveling-scene type, an annotation like “Joseph is traveling” can be generated since the scene type helps narrowing down the search for only the character name in the neighboring scenes since locales are not important. The annotation is likely to be more accurate, and the annotation time is reduced, which will enable more complex queries to be supported.

C. Region Selection and Feature Extraction

In the following, each shot is represented by two keyframes, the first and the last frames of the shot. Other keyframe selection techniques and more keyframes per shot can be used. For each keyframe, a feature vector of five visual features is extracted from two predetermined regions [see Fig. 4(a)]. Each feature is called a *color key*. For MPEG videos, the color key is computed as the average value of all the DC coefficients of the Y color component in the corresponding region. This feature is chosen because the human visual system is more sensitive to luminance, and shots in the same scene are much more visually different than frames within the same shot. Hence, we do not need the more discriminating features such as color histogram or block matching as in ToC or SIM. For uncompressed videos, the color key can be computed using the average pixel values instead of DC coefficients.

As shown in Fig. 4(b), five color keys are extracted from the entire $\square$ background region, the upper left corner (B), the upper right corner (C), the bottom left corner (D), and the bottom right corner (E), respectively. The shape of the region is designed for 1) capturing the essential area of frames according to the strict scene definition and 2) easy feature extraction that does not require complex object segmentation. In Fig. 4(a), region 1, the $\square$ shape background region, is for detecting shots in the same locale. The horizontal bar of the region can detect 1) a common background between consecutive shots when the 180° system is used; and 2) the same background of repetitive shots of the same character in the same locale due to shot/reverse-shot.

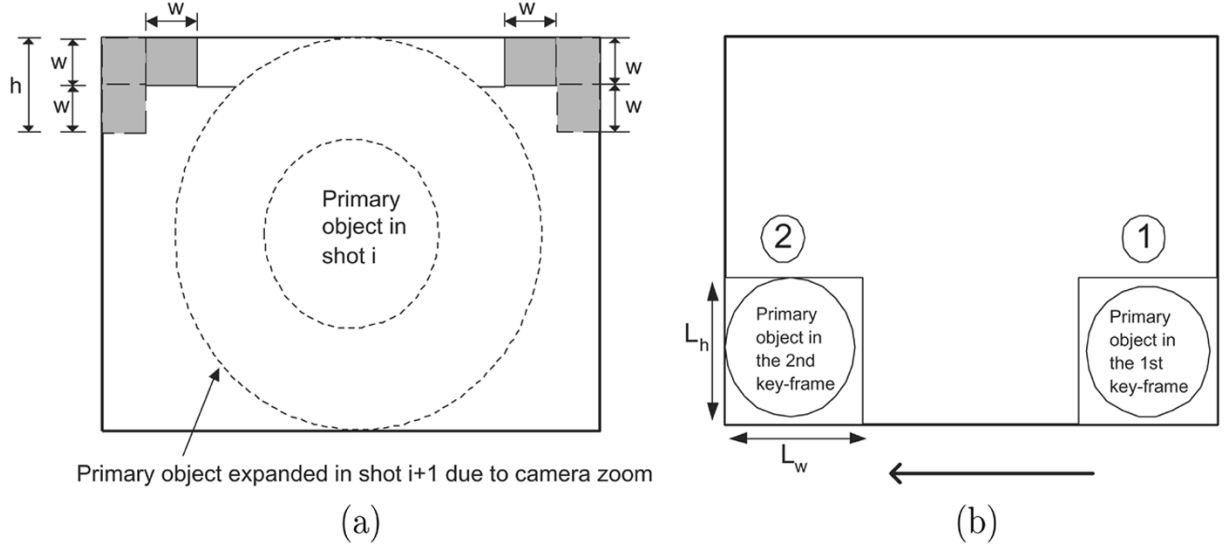


Fig. 5. Detecting different scenarios. (a) Detecting closeups and (b) detecting a traveling event.

The horizontal bar works well when no objects appear in the $\square$ background region and when similar tones of background color are used in the same locale. For instance, in a serial-event scene of a party held in a dining room, since the four walls of the room typically have the same shade of color, when the camera changes its focus from one person in the first shot to the next person sitting along a different wall in the next shot, the background of the two shots is still likely to be similar.

In Fig. 5(a), the two corners of Region 1 detect shots taken in the same locale in the presence of a closeup of an object or an object moving toward a camera. Although the middle of the horizontal bar of the $\square$ background region is disrupted, the two corners in consecutive shots are still likely to be similar since closeups typically occur in the center of the frame.

Region 2 in Fig. 4 consists of the lower left and right corners for detecting a simple traveling event. The main character in each shot in Fig. 5(b) begins at one corner of a frame (either corner) and travels to the other corner in the last frame of the same shot. In the next shot, the same character travels in the same direction to maintain viewers' perception that the character is still traveling. The $\square$ background of these two shots tends to be different because the character travels across different locales, but the two lower corners capturing the character are likely to be similar.

The sizes of the regions are calculated as follows. Let FW and FH be the width and the height of a frame in MPEG blocks, respectively. Let w be the width of the horizontal bar in the $\square$ background region, and h denotes the height of the upper corner. Both w and h are measured in MPEG blocks. The following equations compute the values of L_w , L_h , w , and h shown in Fig. 4(a).

$$L_h = w + h \quad (1)$$

$$h = 2 \cdot w \quad (2)$$

$$2 \cdot L_h = \lceil 0.8 \cdot FH \rceil \quad (3)$$

$$3 \cdot L_w = \lceil 0.9 \cdot FW \rceil. \quad (4)$$

The lower corner ($L_h \times L_w$) is made slightly larger than the upper corner since the lower corners are for capturing the traveling of primary characters whereas the upper corners are to exclude closeup objects. Therefore, in (1), L_h is made about w more than h . In (2), h is chosen to be twice w . The middle area of the frame typically contains many objects, making it sensitive to false shot grouping. Hence, (3) ensures that the upper corner and the lower corner do not meet vertically, and (4) prevents the two lower corners from covering the center bottom area of the frame horizontally.

D. Feature Comparison

To determine the similarity between any two shots, say shots k and m , where $m > k$, feature vectors of all combinations of the keyframes of the shots are considered. That is, if two keyframes per shot are used, features of the first keyframe of shot k are compared to those of the first and of the second keyframes of shot m , and the same is done for the features of the second keyframe of shot k . For each comparison, the following steps are taken.

Background Criterion. If the difference between the color keys of the $\square$ background regions is within 10% of the color key of the $\square$ background region of shot k , the two shots are considered similar due to locale. They are grouped in the same scene, and no other keyframes of these shots are compared. Otherwise, the upper-corner criterion is checked next.

Upper-corner Criterion. Compute the difference of the color keys of the upper left corners and the difference of the color keys of the upper right corners. If the minimum of the two differences is within 10% of the color key of the corresponding corner of shot k , the two shots are considered similar due to locale, and no other feature comparisons of these shots are needed. Otherwise, the lower-corner criterion is checked next. The upper-corner comparison helps detecting the same locale due to closeups.

INPUT: Video file, Frame numbers of the first key-frames of shots (i.e., shot boundaries)
 F : Forward search range; R : Backward search range /* $F \geq R$ */
OUTPUT: Frame numbers of the beginning frames of detected scenes (i.e., scene boundaries)
NOTATIONS: $sl(c)$: Number of shots in scene c
 $CtPreShot$: Number of preceding shots of the current shot
 $CtFutureShot$: Number of subsequent shots from the current shot to the last shot of the video

BEGIN
Step1: /* eliminate very short shots */
Combine a shot less than half a second with its previous shot, if any.
Update the shot boundaries accordingly.
The first shot is the current shot.
DO
Step2: /* forward comparison */
 $r = \min(R, CtPreShot)$; $f = \min(F, CtFutureShot)$;
Find the nearest matching shot within f future shots using the feature comparison in Section III-D.
If a matching shot of the current shot is found,
group the matching pair and the shots in between in the same scene;
the matching shot becomes the current shot, and goto Step2.
Step3: /* backtrack comparison */
 $i = \text{shot number of the current shot}$;
DO
Find the nearest matching shot between shot $i - 1$ and shots $i + 1, i + 2, \dots, i - 1 + f$ using
the feature comparison in Section III-D.
If a matching shot of shot $i - 1$ is found,
group the matching pair and the shots in between in the same scene;
the matching shot becomes the current shot, and goto Step2;
Else $i = i - 1$;
WHILE shot i is still within the r^{th} preceding shot of the current shot;
Step4: /* re-establishing shot check */
The next shot becomes the current shot.
 c is the scene number of the scene of the previous shot.
If $sl(c - 1) == 1$ and $sl(c) > F$,
Compare the current shot with the only shot in scene $c - 1$ (See Fig. 7(a)).
If the two shots are similar based on the background criterion only,
assign the matching pair and the shots in between to scene c ; goto Step5;
Find the nearest matching shot within $\min(\lceil \lg_2 sl(c) \rceil + 1, sl(c))$ preceding shots (see Fig. 7(b)).
If a matching shot is found based only on the background criterion,
assign the current shot to scene c ; goto Step5;
Else declare a scene cut before the current shot; goto Step2;
Step5:
Declare a scene cut right after the current shot; the next shot becomes the current shot.
WHILE the last shot of the video has not been assigned the scene;
END

Fig. 6. Clustering algorithm of ShotWeave.

Lower-corner Criterion. This is similar to the upper-corner criterion, but features from the lower corners are utilized instead. If they are not similar, the two shots are not similar. The lower-corner comparison is used to detect a traveling scene.

The advantage of our feature comparison is that the type of the event and the detected scene can be identified, which is useful for future browsing and annotation generation as mentioned previously. If shot m where $m = k + 1$ is found similar to shot k due to the background comparison, the two shots could represent a noninteracting event or an interacting event in the same locale. If the lower corner is used to group these shots together, the two shots are part of a traveling event. If shot m where $m > k + 1$ is the nearest similar shot to shot k , both shots k and m are highly likely to capture 1) the same character in an interacting event (e.g., in a conversation scene, shots k and m focus on the same person in one location and a shot in between them captures another person possibly in another location) or 2) the same serial-event scene in a parallel-event scene. We note that the 10% threshold is selected in all the three criteria since it consistently gives a better performance than other thresholds in our experiments.

E. Clustering Algorithm

The pseudo-code of our clustering algorithm is shown in Fig. 6. To prevent shots that are too far apart from being miscombined in the same scene, temporal limitations F and R are also used as input parameters. The selection of shots to compare in the forward comparison and the backtrack comparison (Label Step2 and Step3 in Fig. 6) was introduced in SIM [4]. However, the feature extraction, the feature comparison, and the other steps in Fig. 6 are introduced in this paper. We use SIM selection of shots since it is simple to implement and uses a small memory space to store keyframes of only $F + R + 1$ shots for the forward and the backtrack comparison during the entire clustering process. A memory space for one more shot is used for checking the re-establishing shot.

Since a very short shot appears too briefly to be meaningful by itself, it is first combined with its previous shot (Label Step1 in Fig. 6). The very short shot is typically the result of the imperfect shot detection that detects false shot boundaries due to fast camera operations, a sudden brightness such as a flashlight, etc. The forward comparison (Label Step2 in Fig. 6) groups shots into events based on the feature comparison in Sec-

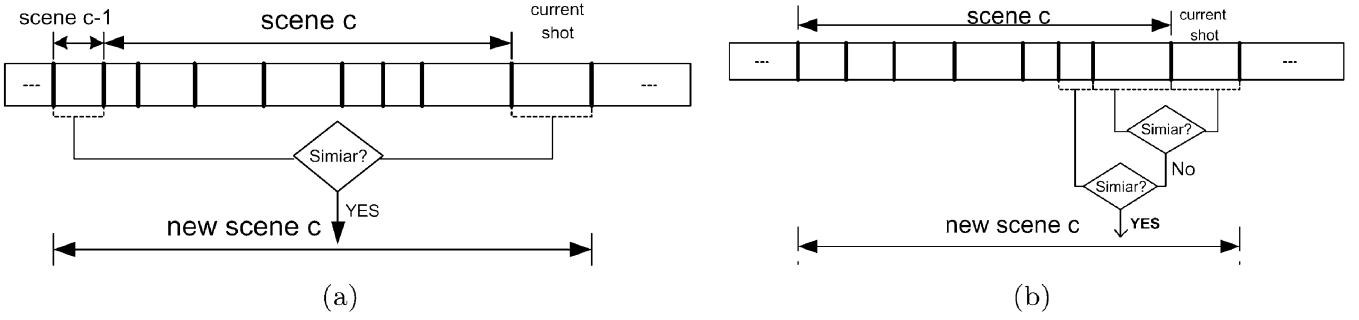


Fig. 7. Scenes with a re-establishing shot. (a) Scene with an establishing shot and (b) scene without establishing shots.

tion III-D. For ease of presentation, we assume that $CtPreShot$ and $CtFutureShot$ are implicitly updated to the correct values as the algorithm progresses. The backtrack comparison (Label Step3 in Fig. 6) is necessary since the pair of matching shots discovered in this step captures 1) another event parallel to the event captured by the forward comparison or 2) a different character in the same interacting event. Note that the feature extraction is done when needed since it can be performed very quickly. The extracted features are kept in memory and purged when the associated shot is no longer needed.

If the forward and the backtrack comparisons fail, the next shot is determined whether it is the re-establishing shot of the current scene (Label Step4 in Fig. 6). In Fig. 7(a), shots in scenes $c-1$ and c , and the current shot are merged into one scene. That is, scene $c-1$ is not the actual scene, but the establishing shot of scene c , and the current shot is the re-establishing shot of the scene. A scene may not have an establishing shot [see Fig. 7(b)]. In this case, the current shot is found similar to one of its preceding shots of the current scene. The lg_2 function used in this step is to reduce the chance of combining shots that are too far apart in the same scene. Unlike the forward and backtrack comparisons, only the background criterion is used for checking the re-establishing shot since the establishing/re-establishing shots are typically more visually different from other shots in the same scene.

Fig. 8 shows an example of a scene boundary detected using ShotWeave. Shots linked by the dark line are similar based on the feature comparison whereas those linked by the gray line are automatically included in the scene. Shots 1, 2, 5, and 8 are grouped together during the forward comparison as follows. The first current shot is shot 1. Shot 2 similar to the current shot becomes the new current shot. Shots 3 and 4 are not similar to shot 2, but are automatically included in the same scene since shot 5, the last shot in the forward search range, is similar to shot 2. Shot 5 becomes the current shot. Since its nearest similar shot is shot 8, shots 6 and 7 are automatically included. Shot 8 becomes the current shot, but no similar shots within the forward search range are found. Thus, the backtrack comparison starts. Shot 7 is checked first and is similar to shot 9. Shot 9 is included in the scene and becomes the current shot. Since no future shots are similar to shot 9 based on the forward comparison and the backtrack comparison also fails, shot 10 becomes the current shot. A scene cut is declared since shot 10 fails the re-establishing shot check.

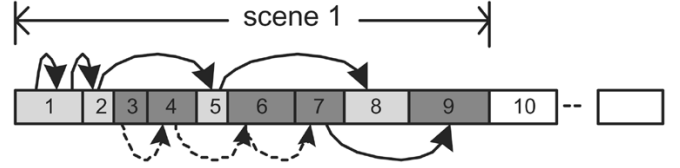


Fig. 8. Example of a detected scene when $F = 3$ and $R = 1$.

IV. EXPERIMENTAL STUDY

In this section, the performance of the two existing techniques and ShotWeave in terms of segmentation accuracy and time is investigated on two test videos, each lasting more than 100 min. Let N_c and N_t be the number of correct scene boundaries and the total scene boundaries detected by a shot clustering technique, respectively. N_t includes both correct scene and false scene boundaries. False boundaries do not correspond to any manually segmented boundaries. N_a denotes the total number of manually segmented scene boundaries. The following metrics are used.

- Recall (H_r) = N_c/N_a . High recall is desirable; it indicates that the technique is able to uncover most scene boundaries judged by humans.
- Precision (H_p) = N_c/N_t . High precision is desirable, indicating that most automatically detected boundaries are correct boundaries.
- Utility (U_m) = $\alpha_r \cdot H_r + \alpha_p \cdot H_p$, where $\alpha_r + \alpha_p = 1$; α_r and α_p are the weights for the recall and the precision, respectively. The values of α_r or α_p are between 0 and 1. Utility measures the overall accuracy of a shot clustering technique, taking both recall and precision into account. Different weights of recall and precision can be used, depending on which measure is more important to the user. In general, techniques offering high utility are more effective. In this study, an equal weight is assigned to recall and precision.
- Segmentation Time (R_t): Time taken in seconds to cluster shots given that shot boundaries have been detected and all needed features have been extracted. After shot detection, each shot has two keyframes with all necessary information for each of the clustering algorithms such as DC values or color histograms of the keyframes.

The implementation of SIM and ToC is based on the information described in the original papers. All the experiments were done on an Intel Pentium III 733 MHz machine running

Linux. In the following, we first study the characteristics of the test videos and experimentally determine the best parameter values for each technique. Finally, three techniques were evaluated to determine the effectiveness of using several region features and the feature comparison in ShotWeave compared to utilizing global features in SIM and ToC.

A. Characteristics of Test Videos

Two MPEG-1 videos converted from the entire *Home Alone* (HA) and *Far and Away* (FA) were used. Each video has the frame rate of 30 frames/s and the frame size of 240×352 pixels. Scenes were segmented manually as follows.

- First, shot boundaries were manually determined. The boundaries were categorized into sharp cuts or gradual transitions of different types (dissolve, fade in, fade out, and wipe). This is to investigate the use of gradual transitions in narrative films.
- Second, shots were manually grouped into scenes according to the strict scene definition. The brief description of the content and the functionality of each shot in a scene (i.e., an establishing shot, a re-establishing shot, etc.) were recorded.

The characteristics of the test videos are summarized in Table III. We did not use the same test videos as in the original papers of SIM and ToC since the video titles were not reported and some of those test videos were only 10–20 min segments of the entire movies. Table III reveals that gradual shot transitions occur in less than one percent of the total number of shots in either movie. The average shot is from 4–5 s in length. Both titles were produced in the 1990s, suggesting that shot clustering techniques are important for newer films than early films³. No temporal relations between shots (i.e., the manipulation of shots of different lengths) to create certain feelings such as suspense were found in the two titles.

The best values of important parameters for each technique were experimentally determined in Sections IV-B–E. This is done by varying the value of the parameter being investigated and fixing the other parameters on video HA. The best parameter values enable the technique to offer high recall and precision.

B. Determining Important Parameter Values for SIM

The three important parameters for SIM are the forward search range (F), the backward search range (R), and the percentage of the best matching blocks to the total blocks in the shot image (B). Table IV shows the results when (F, R) was fixed at (2, 1) and B was varied. The higher the B value is (i.e., more number of blocks of shot images are involved in determining the similarity), the more correct scenes are detected, but the precision drops at a faster rate. The highest utility (0.258) was obtained when B was 20%, the case for the highest precision. However, only two scenes were detected for the entire video, and one of which was a correct scene. Instead, we chose $B = 80\%$ because it gives a reasonable number of correct scenes and offers the second highest utility.

³Early films created around 1895–1905 tend to have shots of longer duration due to lack of effective ways to edit shots at the time [16].

TABLE III
CHARACTERISTICS OF TEST VIDEOS

Video Title	HA	FA
Duration (min:sec)	103:44	132:45
Number of scenes	62	58
Number of shots	1349	1493
Average shot length (in frames)	121	153
Number of abrupt shot cuts	1340	1490
Number of fade in/fade out	1	2
Number of dissolves	6	1
Number of wipes	2	0

TABLE IV
SIM PERFORMANCE WITH $F = 2$ AND $R = 1$ FOR DIFFERENT B VALUES

$B(\%)$	N_c	N_t	H_r	H_p	U_m	R_t
20	1	2	0.016	0.500	0.258	1391
30	1	4	0.016	0.250	0.133	1412
40	5	18	0.081	0.278	0.179	1486
50	8	28	0.129	0.286	0.207	1550
60	9	35	0.145	0.257	0.201	1667
70	13	66	0.210	0.197	0.203	1881
80	18	97	0.290	0.186	0.238	2038
90	13	120	0.209	0.108	0.159	2139

TABLE V
SIM PERFORMANCE WITH $B = 80$ AND $R = 1$ FOR DIFFERENT F VALUES

F	N_c	N_t	H_r	H_p	U_m	R_t
2	18	97	0.258	0.165	0.238	2038
3	13	64	0.210	0.203	0.206	1800
4	12	46	0.194	0.261	0.227	1671
5	7	35	0.113	0.200	0.156	1473
6	8	29	0.129	0.276	0.202	1427
7	9	30	0.145	0.300	0.222	1222
8	9	25	0.145	0.360	0.252	1170

Table V shows the results when F was varied. The highest utility was obtained when F was 8. However, only nine correct scenes ($N_c = 9$) were detected. F of 2 was chosen because it gave the second highest utility with twice as many correctly detected scenes. Because R must be less than F , the best parameter (B, F, R) for SIM was (80, 2, 1).

C. Determining Important Parameter Values for ToC

ToC first assigns shots to groups, and then scenes. The two important parameters for ToC are the group similarity threshold (g_t) and the scene similarity threshold (s_t). Both parameters were suggested to only be determined by the user once and can be used for other videos for the same user. For any two shots to be in the same scene, the similarity between them must be greater than the threshold. To find the best scene threshold, the group threshold was fixed at 1.25 as recommended [18], and the scene threshold was varied. The results are depicted in Table VI. This best scene threshold was later used to determine the best group threshold.

As the scene threshold increases (i.e., shots to be considered in the same scene must be more similar), more scene boundaries are generated, increasing both the number of correct and false scene boundaries. However, the number of correct scene boundaries does not improve further ($N_c = 11$) when the scene threshold is beyond 0.8 whereas the number of false boundaries keeps rising. ToC gives the highest utility (i.e., $U_m = 0.037$)

TABLE VI
ToC PERFORMANCE WHEN $g_t = 1.25$

s_t	N_c	N_t	H_r	H_p	U_m	R_t
0.5	7	254	0.113	0.028	0.070	96
0.6	9	268	0.145	0.034	0.090	96
0.7	9	283	0.145	0.032	0.088	97
0.8	11	298	0.177	0.037	0.107	96
0.9	11	337	0.177	0.033	0.105	96
1.0	11	369	0.177	0.030	0.104	96

TABLE VII
ToC PERFORMANCE WHEN $s_t = 0.8$

g_t	N_c	N_t	H_r	H_p	U_m	R_t
0.9	7	287	0.113	0.024	0.069	66
1.0	8	337	0.129	0.024	0.076	73
1.25	11	298	0.177	0.037	0.107	96
1.6	12	353	0.194	0.034	0.114	149
1.8	12	377	0.194	0.032	0.113	188

when the scene threshold equals 0.8. This value was, therefore, selected as the best scene threshold and used for determining the best group threshold.

Table VII gives the results of ToC using a scene threshold of 0.8 and different group thresholds. When the group threshold is 1.6, the highest utility and recall are achieved. Beyond this threshold, the number of correct scene boundaries is not significantly changed, but the number of false boundaries increases as indicated by a drop in the precision. Hence, the best group and scene thresholds for ToC were 1.6 and 0.8, respectively.

D. Determining Important Parameter Values for ShotWeave

The performance of ShotWeave under different values of F is shown in Table VIII. ShotWeave achieves the best utility when $F = 2$ because of the very high recall of 0.71. In other words, about 71% of the actual scene boundaries are correctly detected by ShotWeave. However, the number of detected scenes is also high. As F increases, recall drops and precision increases. We chose F of 3 instead of 2 since ShotWeave gives the second highest utility with much less number of total scenes detected. When comparing the scene boundaries detected by ShotWeave with the manually detected scene boundaries, it was observed that if F can be dynamically determined based on the number of participating characters in an interacting event, the performance of the technique may be further improved. For instance, if three people participate in an interacting event, F of 2 is too limited because it takes at least three shots, each of which captures each of the persons to convey the idea that these persons are interacting.

To find the best value for R , different values of R were chosen in the experiments, keeping F fixed at 3. The results in Table IX indicate that $R = 2$ offers the highest precision while the same recall was maintained. Thus, (F, R) of $(3, 2)$ was selected as the best parameter values for ShotWeave.

E. Performance Comparison

After selecting the best values for important parameters for each technique, the three techniques are compared. The results are shown in Table X. SIM and ToC do not perform so well on the test videos. Note that both precision and recall are lower than

TABLE VIII
SHOTWEAVE PERFORMANCE WHEN $R = 1$ FOR DIFFERENT F VALUES

F	N_c	N_t	H_r	H_p	U_m	R_t
2	44	428	0.709	0.102	0.406	6.14
3	32	216	0.516	0.148	0.332	6.25
4	25	148	0.403	0.168	0.286	6.32
5	23	106	0.371	0.216	0.293	6.45
6	16	63	0.258	0.253	0.256	6.26
7	14	48	0.225	0.291	0.258	6.5
8	9	37	0.145	0.243	0.194	6.61

TABLE IX
SHOTWEAVE PERFORMANCE WHEN $F = 3$ FOR DIFFERENT R VALUES

R	N_c	N_t	H_r	H_p	U_m	R_t
1	32	216	0.516	0.148	0.332	6.25
2	32	205	0.516	0.156	0.336	7.7

those reported in the original works. This is due to the fact that different test videos and scene definitions were used. Also, in the original evaluation of SIM, if the detected scene boundary was within four shots from the boundary detected manually, this boundary was counted as a correct boundary. In this study, the detected boundary was only counted as correct when it was exactly the same as the boundary of the manually detected scene. In the original evaluation of ToC, the test videos were shorter. The longer the video, the higher the probability that different types of camera motions and filming techniques are used, affecting the effectiveness of the technique.

ShotWeave outperforms existing techniques in all four metrics; it offers at least 2.5 times of the recall and precision of ToC on both test videos. ShotWeave gives twice as much of the recall offered by SIM with the comparable precision. Furthermore, ShotWeave takes much less time than both existing techniques to identify scene boundaries. For ShotWeave, time for feature extraction was also accounted for in the segmentation time whereas the time for feature extraction was not included in SIM and ToC. The short running time of less than 10 s for 1.5 h movies allows ShotWeave to be performed on the fly once the users identify their desirable weights for recall or precision. ShotWeave can be easily extended to output the reasons that shots are clustered together. This information is useful for effective browsing and for adding annotations to each scene.

Nevertheless, when the detected scenes were analyzed, several scenes, each consisting of a single shot were found. These single-shot scenes are, in fact, establishing shots of the nearest subsequent scene. In many cases, these establishing shots are not visually similar to any of the shots in the scene, causing a reduction in the precision of ShotWeave. Another observation is that despite the improvement offered by ShotWeave, the utilities of the three techniques are relatively low due to the complexity of the problem.

V. CONCLUDING REMARKS

Effective shot clustering techniques grouping related shots into scenes are important for content-based browsing and retrieval of video data. However, it is difficult to develop a good shot clustering technique if the scene definition is too broad and very subjective. In this paper, we introduce a strict scene definition for narrative films by defining three scene types based on

TABLE X
PERFORMANCE COMPARISON

Technique	HA						FA					
	N_c	N_t	H_r	H_p	U_m	R_t	N_c	N_t	H_r	H_p	U_m	R_t
SIM	18	97	0.258	0.165	0.238	2038	12	127	0.206	0.094	0.150	2249
ToC	12	353	0.194	0.034	0.114	149	8	418	0.138	0.019	0.078	164
ShotWeave	32	205	0.516	0.156	0.336	7.7	36	249	0.621	0.144	0.383	6.47

three event types. We present a novel shot clustering technique called *ShotWeave* based on the strict scene definition. The crux of *ShotWeave* is 1) the use of features extracted from carefully selected areas of keyframes and 2) the feature comparison based on continuity-editing techniques used in film literature to maintain viewers' thought in the presence of shot breaks.

Given the complexity of the problem, our experimental results indicate that *ShotWeave* performs reasonably well. It is more robust than two recent shot clustering techniques using global features on full-length films consisting of a wide range of camera motions and a complex composition of related shots. Our experience with *ShotWeave* suggests that utilizing visual properties alone will not improve the performance of *ShotWeave* much further. We are investigating the use of sound with *ShotWeave* to improve the segmentation accuracy.

REFERENCES

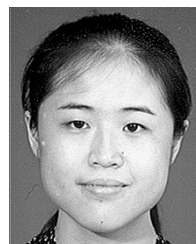
- [1] H. J. Zhang, A. Kankanhalli, and S. W. Smoliar, "Automatic partitioning of full-motion video," *ACM Multimedia Syst.*, vol. 1, no. 1, pp. 10–28, 1993.
- [2] H. J. Zhang, J. H. Wu, D. Zhong, and S. W. Smoliar, "Video parsing, retrieval and browsing: An integrated and content-based solution," *Pattern Recognit., Special Issue on Image Databases*, vol. 30, no. 4, pp. 643–658, Apr. 1997.
- [3] Y. Zhuang, Y. Rui, T. S. Huang, and S. Mehrotra, "Adaptive key frame extraction using unsupervised clustering," in *Proc. Int. Conf. Image Processing*, vol. 1, Chicago, IL, Oct. 1998, pp. 866–870.
- [4] A. Hanjalic, R. L. Lagendijk, and J. Biemond, "Automated high-level movie segmentation for advanced video-retrieval systems," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 9, pp. 580–588, June 1999.
- [5] A. D. Bimbo, *Content-Based Video Retrieval*. San Francisco, CA: Morgan Kaufmann, 1999.
- [6] P. Aigrain and P. Joly, "The automatic real-time analysis of file editing and transition effects and its applications," *Comput. Graph.*, vol. 18, no. 1, pp. 93–103, 1994.
- [7] B. L. Yeo and B. Liu, "Rapid scene analysis on compressed video," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 5, pp. 533–544, 1995.
- [8] T. Shin, J.-G. Kim, H. Lee, and J. Kim, "A hierarchical scene change detection in an MPEG-2 compressed video sequence," in *Proc. IEEE Int. Symp. Circuits and Systems*, vol. 4, Monterey, CA, May 1998, pp. 253–256.
- [9] N. Gamaz, X. Huang, and S. Panchanathan, "Scene change detection in MPEG domain," in *Proc. IEEE Southwest Symp. Image Analysis and Interpretation*, Tucson, AZ, Apr. 1998, pp. 12–17.
- [10] A. M. Dawood and M. Ghanbari, "Clear scene cut detection directly from MPEG bit streams," in *Proc. IEEE Int. Conf. Image Processing and Its Applications*, vol. 1, Manchester, U.K., July 1999, pp. 285–289.
- [11] J. Nang, S. Hong, and Y. Ihm, "An efficient video segmentation scheme for MPEG video stream using macroblock information," in *Proc. ACM Multimedia '99*, Orlando, FL, Nov. 1999, pp. 23–26.
- [12] W. Wolf, "Key frame selection by motion analysis," in *Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing*, vol. 2, Atlanta, GA, May 1996, pp. 1228–1231.
- [13] W. Xiong, J. C.-M. Lee, and R. Ma, "Automatic video data structuring through shot partitioning," *Mach. Vis. Applicat.*, vol. 10, pp. 51–65, 1997.
- [14] A. M. Ferman and A. M. Tekalp, "Multiscale content extraction and representation for video indexing," *Proc. SPIE Multimedia Storage and Archival Systems II*, vol. 3229, pp. 23–31, Oct. 1997.
- [15] A. Girgensohn and J. Boreczky, "Time-constrained keyframe selection technique," in *Proc. Int. Conf. Multimedia and Computing Systems*, vol. 1, Florence, Italy, June 1999, pp. 756–761.
- [16] D. Bordwell and K. Thompson, *Film Art: An Introduction*, 5th ed. New York: McGraw-Hill, 1997.
- [17] J. Oh and K. A. Hua, "Efficient and cost-effective techniques for browsing and indexing large video databases," in *ACM SIGMOD*, Dallas, TX, May 2000, pp. 415–426.
- [18] Y. Rui, T. S. Huang, and S. Mehrotra, "Constructing table-of-content for videos," *ACM Multimedia Syst.*, vol. 7, no. 5, pp. 359–368, September 1999.
- [19] J. M. Corridoni and A. D. Bimbo, "Structured representation and automatic indexing of movie information content," *Pattern Recognit.*, vol. 31, no. 12, pp. 2027–2045, 1998.
- [20] M. M. Yeung and B. Liu, "Efficient matching and clustering of video shots," in *Proc. IEEE Int. Conf. Image Processing*, vol. 1, Washington, DC, Oct. 1995, pp. 338–341.
- [21] T. Lin and H.-J. Zhang, "Automatic video scene extraction by shot grouping," in *Proc. 15th Int. Conf. Pattern Recognition*, vol. 4, Barcelona, Spain, Sept. 2000, pp. 39–42.
- [22] E. Veneau, R. Ronfard, and P. Bouthemy, "From video shot clustering to sequence segmentation," in *Proc. 15th Int. Conf. Pattern Recognition*, vol. 4, Barcelona, Spain, Sept. 2000, pp. 254–257.
- [23] H. Sundaram and S. F. Chang, "Determining computable scenes in films and their structures using audio-visual memory models," in *Proc. ACM Multimedia '00*, Los Angeles, CA, Oct. 2000, pp. 95–104.
- [24] —, "Video scene segmentation using audio and video features," in *Proc. IEEE ICME 2000*, vol. 2, New York, July 2000, pp. 1145–1148.
- [25] B. Adams, C. Dorai, and S. Venkatesh, "Novel approach to determining tempo and drama story sections in motion pictures," in *Proc. IEEE ICME 2000*, New York, July 2000, pp. 283–286.



Wallapak Tavanapong (S'95–M'99) received the B.S. degree in computer science from Thammasat University, Thailand, in 1992 and the M.S. and Ph.D. degrees in computer science from the University of Central Florida, Orlando, in 1995 and 1999, respectively.

Since Fall 1999, she has been a faculty member of the Department of Computer Science at Iowa State University, Ames. Her current research interests include high-performance multimedia servers, distributed multimedia caching systems, multimedia databases, ontology, and semi-structure data.

Dr. Tavanapong is the recipient of the NSF CAREER award in 2001. She has served as an editorial board member for ACM SIGMOD Digital Symposium Collection, a program committee member for international conferences, and a referee for several conferences and journals. She is a member of the ACM.



Junyu Zhou received the B.S. degree in acoustics in 1993 and the M.S. degree in signal processing in 1996, both from Nanjing University of Science and Technology, China, and the M.S. degree in computer science from Iowa State University, Ames, in 2001.